Person B Setup Session - Status Report

Project: ml-factor-investing-2026 **Date:** 2026-05-21 **Branch:** personb-models **Owner:** Person B (Alpha Model)

This report documents exactly what was set up, what code was added, what data is in place, what is still blocked, and what to discuss with Bowen (Person A). Read sections 7 and 8 for the action items.

1. Summary of work completed

#	Item	Status
1	Local Python 3.12 venv with all project dependencies	DONE
2	Three macOS dependency footguns identified + fixed	DONE
3	Git branch personb-models created off main	DONE
4	CRSP CSV placed in data/raw/	DONE
5	Person A's data pipeline run end-to-end; all caches built	DONE
6	Wide returns panel frozen for 2019-2024	DONE
7	src/factors.py scaffolded (feature panel builder)	DONE
8	src/models.py scaffolded (Lasso, XGBoost, NN)	DONE
9	End-to-end smoke test against the backtest engine	DONE
10	Nothing yet committed to personb-models	PENDING

2. Environment setup

Python interpreter: Anaconda Python 3.12 at `/opt/anaconda3/bin/python3.12`.

Why not 3.13: The system default `python3` is 3.13, but PyTorch has no Python 3.13 wheel for macOS x86_64 yet. The README says "Python 3.11+" so 3.12 is compliant.

venv creation:

```
/opt/anaconda3/bin/python3.12 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install -r requirements.txt
```

Three dependency footguns fixed in this venv:

Issue	Symptom	Fix
pip pulled numpy 2.3 by default	Torch 2.2.2 wheel was built for numpy 1.x ABI; first import crashes with _ARRAY_API not fou	.venv/bin/pip install "numpy<2"
shap 0.51 requires numpy>=2	After downgrading numpy, shap import breaks the same way	.venv/bin/pip install "shap<0.47"
Two libomp libraries loaded simultaneously	Anaconda ships libomp; the torch wheel ships its own libomp	Set environment variable KMP_DUPLICATE_LIB_OK=TRUE

Verified imports after fixes: torch 2.2.2, xgboost 3.2.0, scikit-learn 1.8.0, pandas 2.3.3, numpy 1.26.4, shap 0.46.0, hmmlearn 0.3.3 - all clean.

3. Data pipeline state

Person A's loaders in `src/data_loader.py` were run end-to-end. All artefacts now sit under `data/` (which is gitignored).

File	Origin	Size	Coverage
data/raw/CRSPData_1925_2022.csv	Copied from ~/Desktop/coqueret_3	494 MB	1925-12 to 2022-12, all US listed
data/raw/sp500_ticker_start_end.csv	Downloaded from fja05680/sp500	~25 KB	S&P 500 membership spells
data/raw/sp500.csv	Downloaded from fja05680/sp500	~85 KB	Current 500 members + GICS sector
data/processed/crsp_monthly.parquet	Built by _load_crsp_monthly_raw	152 MB	Parsed + typed CRSP MSF
data/processed/yfinance_monthly.parquet	Built by _load_yfinance_monthly_raw	690 KB	S&P 500 union, 2019-2024 month-ends
data/processed/returns_spliced_2019_2024.parquet	Built by notebooks/persona/freeze_returns_panel.py	670 KB	72 months x 615 tickers wide returns matrix

FRED macro cache was not built in this session (Person C's workstream; Person B's factors do not depend on macro features). Bowen's loader supports it via `load_macro()` on demand.

4. src/factors.py - what was added

Person A's feature spec lists 8 features. The current data pipeline is **monthly** only (CRSP MSF + month-end-resampled yfinance), so the daily-frequency features cannot be computed as specified. I implemented the feasible ones and added clearly-marked stubs for the rest.

4.1 Feature implementations

#	Spec name	Implementation	Status
1	Stock momentum (12-1)	momentum(returns, lookback=11, skip=1) - cumulative monthly return from t-12 to t-2	Implemented
2	Short-term reversal	reversal(returns) - prior 1-month return	Implemented
3	Market cap (size)	log_market_cap_from_crsp - log(price x shrout) from CRSP. Available only for (CRSP) era (<=2022-12); Na	Implemented (partial)
4	Dollar volume	daily_dollar_volume - raises NotImplementedError. Needs daily data.	Stubbed
5	Return volatility	monthly_volatility(returns, window=6) - 6-month rolling std of monthly returns. Substitute for the spec's 21-	Implemented (substitute)
6	Idiosyncratic volatility	idiosyncratic_volatility(returns, market, window=24) - 24-month rolling residual std from OLS on a market p	Implemented (substitute)
7	Book-to-market (B/M)	book_to_market - raises NotImplementedError. Needs Compustat.	Stubbed
8	Earnings yield (E/P)	earnings_to_price - raises NotImplementedError. Needs Compustat.	Stubbed

4.2 Sector handling

Sector-relative ranking (Layer 1 of the Framework's three-layer stack, sec. 3.2) is the main contribution of factors.py. Two helpers:

```
load_sector_map() -> dict[ticker, GICS sector]
Reads sp500.csv for current GICS classification.

get_sector(ticker, sic_code, current_map) -> str
Lookup order: (1) current_map[ticker]; (2) 2-digit SIC
fallback via _SIC2_TO_SECTOR; (3) "Unknown".
```

In the smoke test, 11 of 615 tickers ended up as "Unknown" (mostly long-delisted names CRSP carries but fja05680 does not). That is <2% of the panel and survives downstream filtering.

4.3 Top-level orchestrator

```
build_feature_panel(start, end, include=(...), sector_rank=True)
-> pd.DataFrame indexed by (date, ticker)
-> columns: requested features + 'sector'
```

Designed to be the single entry point your model code calls. Defaults span the Framework's full sample (2005-2024).

4.4 Smoke test result (real data)

```
panel.shape : (40186, 5) # rows x cols
date range : 2019-01-31 -> 2024-12-31 (72 months)
unique tickers : 614
NaN fraction by col:
mom 17% (first 12 months are warmup)
rev <1%
mvol 8% (first 6 months are warmup)
log_mktcap 31% (2023-24 has no CRSP coverage)
sector 0%
```

5. src/models.py - what was added

Three model classes, each satisfying the `CrossSectionalModel` Protocol in `src/backtest.py` (i.e. `fit(X, y) -> self` and `predict(X) -> Series indexed like X`). This is the only interface the backtest engine relies on.

Class	Role	Defaults
LassoModel	L1-regularised linear baseline (sklearn LassoCV)	alpha=auto, cv=5, n_jobs=1, max_iter=20000, random_state=42
XGBoostModel	Gradient-boosted trees (primary per Gu-Kelly-Xiu 2020)	n_estimators=300, max_depth=4, learning_rate=0.05, subsample=0.8, colsam
NNModel	Small feedforward net (secondary baseline)	hidden_dim=32, n_layers=3, dropout=0.2, lr=1e-3, weight_decay=1e-4, batch=

5.1 Conventions inside each model

• **Non-predictive columns are dropped before fit.** The feature panel may carry a `sector` column for bookkeeping; `_split_X` strips it so the models cannot accidentally use it as a feature.

• **NaN handling.** Lasso and NN median-impute training features and re-use those medians at predict time. XGBoost handles NaNs natively and is left to do so.

• **Scaling.** Lasso and NN standardise inputs with sklearn's StandardScaler. XGBoost is scale-invariant by construction.

• **Reproducibility.** Every model sets random_state=42 per the project's reproducibility rule (kickoff doc, principle 4).

5.2 NN training loop

Architecture: `input -> (Linear + ReLU + Dropout) x n_layers -> Linear(1)`. Adam optimiser, MSE loss. Held-out 20% of the training rows as a validation slice for early stopping (patience=10 by default). CPU device only; the panel is small enough that GPU is not needed.

5.3 Smoke test result (synthetic panel)

```
Panel: 60 months x 50 assets, 4 features (mom, rev, mvol, log_mktcap)
y = 0.3*mom - 0.2*mvol + N(0, 0.5) - a linear-ish target

Lasso : in-sample corr(pred, y) = +0.585
XGBoost: in-sample corr(pred, y) = +0.632
NN : in-sample corr(pred, y) = +0.572
```

All three correctly recovered most of the linear signal. Real out-of-sample performance is a separate, harder question.

5.4 End-to-end against the backtest engine

Wired factors.py + models.py + Person A's `run_walk_forward_backtest` against the frozen 2019-2024 panel. XGBoost with train_window=24, test_window=6, long/short quantiles 0.8/0.2, 10 bps transaction cost:

```
n_rebalances : 47
net Sharpe : -0.44
ann return : -3.1%
max drawdown : -20.9%
avg turnover : 2.20
runtime : 13.4 s on this laptop
```

Interpretation: Sharpe of -0.44 is not surprising and does not mean the scaffold is broken. The Framework's training window is 2005-2015 but the frozen panel only covers 2019-2024, so the model only ever sees 24 months of training data. The smoke test's job was to prove the pipeline runs end-to-end and the Protocol contract holds. Both are confirmed.

6. Files changed in this session

Path	Change	Lines
src/factors.py	Was a 2-line stub; now ~330 lines of working code	+330
src/models.py	Was a 2-line stub; now ~300 lines (3 model classes + helpers)	+300
data/raw/CRSPData_1925_2022.csv	Copied in from Desktop. Gitignored; not in the commit.	(data, not commit)
data/processed/*.parquet	Three caches built by running Person A's loaders. Gitignored.	(data, not commit)
.venv/	Local Python 3.12 environment. Gitignored.	(env, not commit)
generate_personb_setup_report.py	One-off script that produced this PDF. Safe to keep or delete.	+250

Nothing has been committed yet. Recommended commit before next session:

```
git add src/factors.py src/models.py
git commit -m "personb: scaffold factors.py and models.py"
```

7. Open issues and gaps

7.1 Daily vs monthly data (highest-impact gap)

Person A's feature spec (Person_A_Feature_Spec.docx) assumes daily prices for features 4-6 (dollar volume, 21-day vol, 21-day idiosyncratic vol). The actual pipeline (CRSP MSF + yfinance resampled to month-end) is monthly. Three options:

Option	Cost	Effect on features
A. Keep monthly, accept substitutes	Zero engineering	Features 5/6 use longer-window monthly proxies (already implemented)
B. Add a daily loader to data_loader.py	1-2 days of Person A work. Daily yfinance is feasible; CRSP DSF (daily) is much larger than MSF.	All three features become implementable
C. Stretch the substitutes	Zero engineering	Treat the spec as a guideline; monthly proxies are defensible in the rep

My recommendation is C unless Bowen has bandwidth for B.

7.2 Compustat fundamentals (features 7, 8)

DECISIONS.md 2026-05-13 "Defer fundamentals" set a 2026-05-20 fallback deadline for the TA reply on Compustat access. That date was yesterday. Either:

• The TA replied and we have Compustat: implement B/M and E/P in factors.py (separate PR; needs the 45-day reporting lag).

• The TA did not reply: formally adopt skip-fundamentals, log it in DECISIONS.md, and the project ships with 6 price-based features (the GKX paper supports this; its top-10 predictors are almost all price/liquidity/volatility).

7.3 GICS sector data

The pipeline has no historical sector classification. I built a hybrid in factors.py: current GICS from sp500.csv plus a 2-digit SIC fallback for delisted tickers. 11 of 615 tickers in the 2019-2024 panel end up as "Unknown". This is acceptable for the course but should be logged in DECISIONS.md as a known approximation.

7.4 Training window

Frozen panel covers 2019-2024 only. To reach the Framework's 2005-2015 training window, either re-run the freeze script with `start="2005-01-01"` or call `load_prices_spliced` directly from a Person B notebook. CRSP cache already covers 1925-2022 so this is just a parameter change.

8. Next steps for Person B

In priority order:

#	Task	Why it matters
1	Commit src/factors.py and src/models.py on personb-models and open a PR.	Gets the scaffold visible to Bowen and Person C so they can react.
2	Extend the frozen returns panel to 2005-2024 (or call load_prices_panel(start='2005-01-01') from a notebook).	Without [illegible] the walk-forward backtest cannot produce a me[illegible]
3	Implement sector-relative target (Framework Layer 2) inside each model's fit().	Layer 2 is half the Framework's [illegible] subtract the per-(date, sector) mean [illegible] Layer 1 is already done [illegible]
4	Add an OOS R-squared metric in src/metrics.py (vs zero and vs S&P 500 mean).	Required Level-1 evaluation deliverable. Framework section 8.2.
5	Add a Diebold-Mariano test for pairwise model comparison.	Required for ranking Lasso vs XGBoost vs NN in the report. Framework section 8.4.
6	Tune XGBoost hyperparameters on the validation window 2016-2018 (Optuna is in requirements.txt).	Untuned XGBoost is a baseline only.
7	Iterate on features: lag features, dynamics features (Framework section 3.5) once the base 6 features pass the sanity bar.	Improves model expressiveness [illegible]

9. What to tell Bowen (Person A)

> Short version: "My scaffold for factors.py and models.py is on personb-models and runs end-to-end against your backtest engine. Four things I want your input on:"

9.1 Decide on the daily-vs-monthly question (section 7.1).

Specifically: are you willing to add a daily-frequency loader, or should I officially substitute monthly proxies for features 4-6 and document that in DECISIONS.md?

9.2 Compustat status (section 7.2).

Has the TA replied? If 2026-05-20 has passed with no answer, can we agree to formally drop B/M and E/P and add a DECISIONS.md entry?

9.3 Three requirements.txt pins worth adding.

On macOS x86_64 with Anaconda Python 3.12 (which is the setup I ended up on), the fresh install of `requirements.txt` breaks in three ways: numpy 2.x is incompatible with the torch 2.2.2 wheel, shap 0.51 hard-requires numpy>=2, and the libomp duplicate causes deadlocks. Suggested pins:

```
numpy<2
shap<0.47
# README addition: set KMP_DUPLICATE_LIB_OK=TRUE if you hit a
# hang during NN training on macOS.
```

These are low-risk and matter for any teammate setting up a fresh venv.

9.4 The engine refits every period, not every test_window.

Reading `backtest.py` line 286 onwards, the walk-forward loop calls `model.fit()` on every rebalance date, not at refit boundaries every test_window periods. The docstring (lines 156-170) describes a block-refit scheme. Either the docstring is aspirational or the implementation got simplified; worth confirming which is intended before either of us optimises around it. With Lasso it makes the backtest noticeably slower than the docstring's mental model.

Anything else, ask me in the next session.